

Algorithm Analysis

- A **problem** for computer must be defined precisely by its input and desired output.
- Ex: sort an array
 - Input: array and a way to compare the elements
 - Output: Sorted array
- Ex: Compute the factorial of a positive integer
 - Input: a positive integer n
 - Output: $n!$
- A **solution** is some method of taking an arbitrary input and computing the output with the desired properties defined by the problem.
- An **Algorithm**, is a sequence of steps you can perform to get the desired output from the input.

Note that we can use the tools of discrete math to define these inputs and outputs precisely

Sort(A)

Let S = the set of all permutations of A

For x in S :

 if x is sorted:

 return x

This is an example of a pseudo code. This is a way of communicating an algorithm to another human. It is written in a way that is easier to understand by a human. It has a mix of precise and unambiguous notations and words.

1. Is this algorithm correct?
2. Does this algorithm work efficiently? (runtime)

How to measure the runtime?

1. Idea 1: Implement the algorithm, run it and then time it.
 - Depends on the software, hardware, operating system, programming language, etc.
 - Implementation takes time and money.
 - How to determine which input to run?
 - How do we know the inputs that we selected to run reflect the runtime?
2. Idea 2: Find a function that reflects the runtime in terms of the input size
 - Runtime: number of primitive operations (arithmetic operations, logical operations, variable retrieval, variable assignment, etc.)

• Algorithm 1

Sum = 0 ←

variable

input : n

assignment (1)

for(int i=1; i ≤ n; i++)

For i = 1 to n:

sum = sum + 1

(3 ops)

1 + n(1+3)

{ sum = sum + 1

For i = 1 to n:

sum = sum + 1

} 1 + (1+3)n

} =

For i = 1 to n:

sum = sum + 1

} 1 + (1+3)n

(1) int i = 0
while(i ≤ n) {

$$f(n) = 1 + (1 + (1+3)n)3$$

$$= 1 + 3 + 12n$$

} { sum = sum + 1
i++;
}

• Algorithm 2

Sum = 0

If $n < 100$:

For $i = 1$ to n :

For $j = 1$ to n :

$\underline{\text{sum}} = \underline{\text{sum}} + \underline{n/3}$

Else

$\underline{\text{sum}} = \underline{3n}$

$f_2(n) =$

$\begin{cases} 1 + 1 + n(1 + 6n) \\ 1 + 3 = 4 \end{cases}$

if $n < 100$

otherwise

(1)

variable access + logical operation (2)

$1 + n(1 + (1+5)n)$

$1 + (5+1)n$

5 ops

3 ops

Given $f_1(n) = 15n + 4$

$$f_2(n) = \begin{cases} 9n^2 + n + 3 & n < 10 \\ 6 & n \geq 10 \end{cases}$$



